

ETL Failure Auto-Fix Bot – Project Summary

Problem

Modern data pipelines frequently encounter ETL failures due to schema changes, missing data, query errors, or infrastructure issues. These failures often require manual debugging by data engineers, leading to:

- Increased downtime and delayed data availability
- High operational overhead
- Lack of visibility into recurring failure patterns
- Slow root cause identification and resolution

Teams struggle to scale ETL monitoring and remediation efficiently, especially in complex data environments.

Solution

We built an **AI-powered ETL Failure Auto-Fix Bot** using Snowflake and Streamlit that:

- Automatically ingests ETL failures from query history/Airflow Logs
- Uses **Snowflake Cortex (mistral-large2)** to analyze failures
- Identifies root causes and suggests SQL/python-based fixes
- Enables **human-in-the-loop approval** before applying fixes
- Tracks fix history and outcomes for auditability
- Provides a **chat interface** to query failures in natural language
- Sends **email notifications** for unresolved failures

This transforms ETL operations from reactive debugging to proactive, intelligent automation.

Tech Stack

- **Frontend:** Streamlit
- **Backend / Data Platform:** Snowflake (Snowpark)
- **AI/LLM:** Snowflake Cortex (mistral-large2)

- **Database:** Snowflake tables
 - **Automation:** Stored Procedures & SQL
 - **Notifications:** Snowflake Email Integration
-

Results / Impact

- 🕒 **Significant reduction in debugging time** (manual → AI-assisted)
- ⚡ Faster identification of root causes
- 🤖 Automated fix suggestions improve engineer productivity
- 🔍 Improved visibility into failure trends and severity
- 🛡️ Safe execution via human approval workflow
- 💬 Natural language querying simplifies monitoring

Overall, the solution reduces operational burden and improves ETL reliability.

Next Steps

- Improve fix accuracy with feedback loops
- Integrate with **Slack / Microsoft Teams alerts**
- Implement **role-based approval workflows**
- Enhance AI with **historical learning & pattern detection**
- Add **pipeline-level health dashboards**